FSBM
FACULTE DES SCIENCES BEN M'SICK
UNIVERSITÉ HASSAN II DE CASABLANCA

MATHÉMATIQUES & INFORMATIQUE

RAPPORT DE PROJET — MODULE CLOUD & DEVOPS

Master DSBD & Intelligence Artificielle

Infrastructure DevOps Automatisée

Déploiement Continu d'une Application Quiz Interactive

Réalisé par : CHAIMAA AFESS
KHANSAA BALAKRAFAS
SALMA ZAMAKHCHARI
LINA AKHOUZINE

Année académique : 2025 / 2026

Encadrée par : Prof.Ben Abou

Table des Matières

Table des Matières 2
I. Introduction et Contexte du Projet 4
Problématique 4
Réponse apportée 4
II. Architecture du Projet 5
Description des trois environnements 6
1. Environnement Local (PC du développeur) 6
2. Environnement CI/CD & Registry (GitHub) 6
3. Infrastructure Cloud AWS (us-east-1) 6
III. Livrables Techniques 7
a. Code Source et Conteneurisation (Docker) 7
Structure du projet 7
Dockerfile 7
Choix techniques justifiés 7
b. Pipeline CI/CD — GitHub Actions 9
Déclenchement intelligent 9
Étapes du pipeline 9
Points forts du pipeline 9
c. Infrastructure as Code — Terraform et Ansible 11
1. Terraform — Provisionnement AWS 11
2. Ansible — Configuration des nœuds 12
d. Orchestration Kubernetes — Manifests YAML 14
Manifest Kubernetes complet 14
Analyse des choix techniques 14
Vérification du cluster 15
e. Monitoring et Observabilité 16
1. Contraintes techniques et justification 16
2. Monitoring natif déployé 16
3. Architecture cible (Roadmap Production) 16
IV. Outils et Justification des Choix Technologiques 17
V. Preuves de Fonctionnement 18
1. Infrastructure AWS — Instances EC2 18
2. Cluster Kubernetes (K3s) 18
3. Pipeline CI/CD — GitHub Actions 18
4. Image Docker — Docker Hub 19

5. État du cluster (kubectl) ... 20
VI. Guide de Reproduction (Runbook) ... 22
Prérequis ... 22
Étape 1 — Provisionnement de l'Infrastructure (Terraform) ... 22
Étape 2 — Configuration du Cluster (Ansible) ... 22
Étape 3 — Configuration CI/CD (GitHub Actions) ... 23
Étape 4 — Accès à l'Application ... 23
Étape 5 — Nettoyage des Ressources ... 23
VII. Documentation et Debugging ... 24
Validation avec terraform plan ... 24
Commandes de diagnostic Kubernetes ... 24
Problèmes rencontrés et solutions apportées ... 24
VIII. Conclusion ... 25
Compétences acquises ... 25
Perspectives d'amélioration ... 25
Annexes ... 26
Annexe A — Liens et Ressources ... 26
Annexe B — Récapitulatif des Fichiers Livrés ... 26

I. Introduction et Contexte du Projet

Ce rapport présente la mise en place d'une infrastructure DevOps complète, réalisée dans le cadre du projet final du module Cloud & DevOps du Master DSBD & Intelligence Artificielle à la Faculté des Sciences Ben M'Sick (Université Hassan II de Casablanca). L'objectif est de simuler un environnement de production réel, de bout en bout, en appliquant les principes modernes de l'automatisation, de la conteneurisation et de l'orchestration.

Le projet repose sur le déploiement automatisé d'une application web interactive — une API de Quiz développée en Python/Flask — sur une infrastructure cloud AWS entièrement provisionnée et configurée par du code (Infrastructure as Code). L'ensemble de la chaîne, de la création des serveurs cloud jusqu'à la mise à jour automatique du code applicatif, est codifié, versionné et reproductible sans aucune intervention manuelle.

> **Objectif pédagogique**
> *Maîtriser les pratiques DevOps modernes : Infrastructure as Code (IaC), CI/CD automatisé, orchestration par conteneurs et monitoring, dans un contexte cloud réel.*

Problématique

Comment automatiser intégralement le cycle de vie d'une application — de l'écriture du code jusqu'à son exposition aux utilisateurs finaux — tout en garantissant la reproductibilité, la stabilité et la rapidité des déploiements sur une infrastructure cloud à coût maîtrisé ?

Réponse apportée

- Provisionnement automatique de l'infrastructure AWS via Terraform.
- Configuration des nœuds (Docker, K3s) via Ansible.
- Pipeline CI/CD GitHub Actions pour build, push et déploiement.
- Orchestration Kubernetes (K3s) avec haute disponibilité (2 réplicas).
- Monitoring natif via kubectl et GitHub Actions Metrics.

II. Architecture du Projet

L'architecture du projet repose sur trois environnements distincts et complémentaires, communiquant entre eux de manière automatisée. Le schéma ci-dessous illustre l'ensemble des flux et des composants déployés.

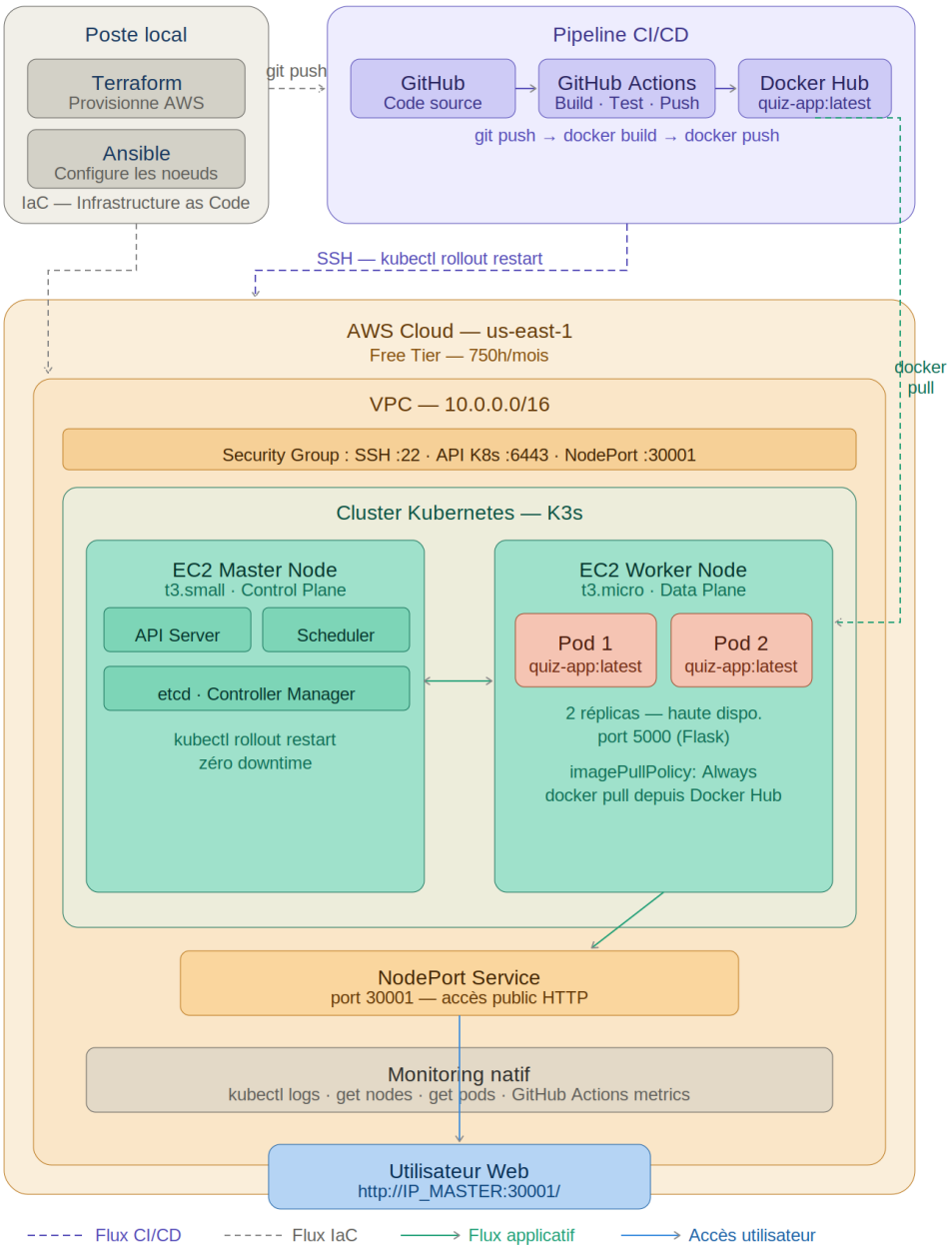


Figure 1 — Architecture globale de l'infrastructure DevOps

Ce schéma présente l'architecture complète du projet, organisée en trois zones distinctes. À gauche, le poste local du développeur héberge Terraform (provisionnement AWS) et Ansible (configuration des nœuds). Au centre, le pipeline CI/CD GitHub Actions orchestre le build de l'image

Docker, son push vers Docker Hub, et le déploiement sur le cluster via SSH. À droite, l'infrastructure AWS regroupe un VPC sécurisé, deux instances EC2 formant un cluster K3s (Master + Worker), et un service NodePort exposant l'application sur le port 30001.

Description des trois environnements

1. Environnement Local (PC du développeur)

Le poste de travail local héberge les outils d'Infrastructure as Code. Terraform est utilisé pour provisionner les ressources AWS depuis la ligne de commande, et Ansible pour configurer les machines une fois créées. C'est également depuis ce poste que le code source est versionné et poussé vers GitHub.

2. Environnement CI/CD & Registry (GitHub)

GitHub centralise le code source du projet. À chaque commit sur la branche main, GitHub Actions déclenche automatiquement le pipeline de CI/CD : construction de l'image Docker, tests, push vers Docker Hub, et déploiement sur le cluster AWS via SSH.

3. Infrastructure Cloud AWS (us-east-1)

Deux instances EC2 Ubuntu 24.04 LTS constituent le cluster Kubernetes K3s : un nœud Master (t3.small) qui héberge le Control Plane (API Server, Scheduler, etcd, Controller Manager), et un nœud Worker (t3.micro) qui exécute les Pods applicatifs. L'application est exposée sur le port 30001 via un service NodePort.

Composant	Rôle	Technologie
Master Node	Control Plane Kubernetes	EC2 t3.small — K3s
Worker Node	Exécution des Pods applicatifs	EC2 t3.micro — K3s Agent
VPC	Réseau isolé AWS	Terraform
Security Group	Pare-feu (ports 22, 80, 6443, 30001)	Terraform
CI/CD Pipeline	Build, Test, Deploy automatique	GitHub Actions
Docker Registry	Stockage des images conteneurisées	Docker Hub
Monitoring	Surveillance cluster et pipelines	kubectl + GH Actions

III. Livrables Techniques

a. Code Source et Conteneurisation (Docker)

L'application est une API REST développée en Python avec le framework Flask. Elle propose 18 catégories de quiz et expose ses routes sur le port 5000. L'interface utilisateur est servie via un template HTML intégré.

Structure du projet

```
projet-devops/
├── app/
│   ├── app.py              ← Logique backend Python/Flask
│   ├── requirements.txt    ← Dépendances (flask)
│   ├── Dockerfile          ← Configuration de l'image
│   └── templates/
│       └── index.html      ← Interface utilisateur
├── terraform/
│   └── main.tf             ← Provisionnement AWS
├── ansible/
│   ├── hosts.ini           ← Inventaire des machines
│   └── setup.yml           ← Playbook de configuration
├── k8s/
│   └── deployment.yaml     ← Manifests Kubernetes
└── .github/
    └── workflows/
        └── deploy.yml      ← Pipeline CI/CD
```

Dockerfile

Le Dockerfile utilise une image de base légère python:3.9-slim pour minimiser la taille de l'image et les surfaces d'attaque. Les dépendances sont installées via pip et le port 5000 est exposé.

```
FROM python:3.9-slim
# Définir le répertoire de travail
WORKDIR /app
# Copier et installer les dépendances
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
# Copier le code source
COPY . .
# Exposer le port Flask
EXPOSE 5000
# Démarrer l'application
CMD ["python", "app.py"]
```

Choix techniques justifiés

- **Image de base :** Image python:3.9-slim

  - Légèreté (environ 125 MB vs 900 MB pour python:3.9), moins de vulnérabilités, démarrage plus rapide sur K3s.
- **Port Flask :** Port 5000
  - Port standard de Flask, mappé vers NodePort 30001 côté Kubernetes.
- **pip install :** --no-cache-dir
  - Évite de stocker le cache pip dans l'image, réduisant encore sa taille.

b. Pipeline CI/CD — GitHub Actions

Le pipeline CI/CD est au cœur de l'automatisation du projet. Configuré dans le fichier .github/workflows/deploy.yml, il garantit que toute modification du code est automatiquement testée, conteneurisée et déployée sur le cluster Kubernetes AWS sans intervention humaine.

Déclenchement intelligent

Le pipeline est optimisé pour ne s'activer qu'en cas de modifications dans les répertoires /app ou /k8s, évitant ainsi des builds inutiles qui consommeraient des ressources GitHub Actions.

Étapes du pipeline

Étape	Action	Détail technique
1. Checkout	Récupération du code	actions/checkout@v3
2. Docker Login	Authentification Docker Hub	Secrets DOCKER_USERNAME / DOCKER_PASSWORD
3. Docker Build	Construction image	docker build --no-cache → tag unique
4. Docker Push	Publication image	Push vers chaimaa00dsbd/quiz-app:latest
5. SSH Deploy	Déploiement cluster	SSH vers EC2 Master via clé privée
6. Rollout	Mise à jour Pods	kubectl rollout restart — zéro downtime
7. Vérification	Contrôle post-deploy	Extraction Pod name + vérif. HTML

Points forts du pipeline

- Fraîcheur garantie : docker build --no-cache assure que les dernières modifications sont intégrées.
- Zéro downtime : kubectl rollout restart met à jour les Pods de manière glissante sans interruption de service.
- Sécurité : les credentials (Docker Hub, AWS SSH) sont stockés en tant que GitHub Secrets chiffrés.
- Vérification automatique : le pipeline extrait dynamiquement le nom du Pod et vérifie le contenu HTML après déploiement.
- Efficacité : déclenchement ciblé sur /app et /k8s uniquement, économisant les minutes GitHub Actions.

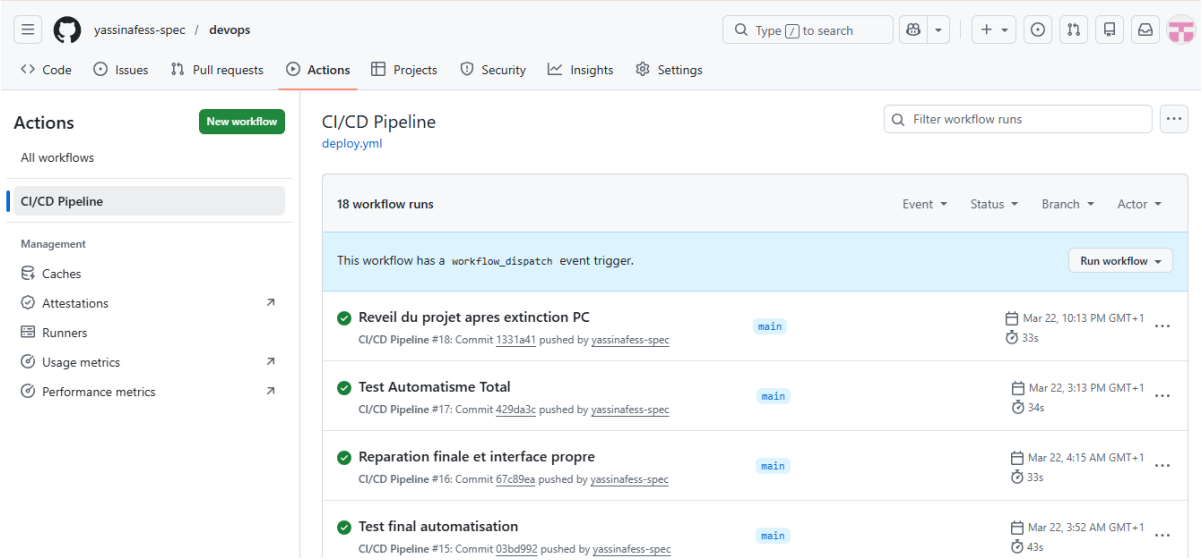


Figure 2 — Succès du pipeline CI/CD : Build, Push et Déploiement Kubernetes automatiques

Cette capture montre le tableau de bord GitHub Actions avec le statut d'exécution du pipeline. Chaque étape est représentée par un badge coloré : vert pour les étapes réussies. On distingue clairement les quatre phases principales — Checkout du code, Build et Push de l'image Docker vers Docker Hub, connexion SSH au Master EC2, et déploiement via kubectl rollout restart. Le temps d'exécution total confirme l'efficacité du pipeline (moins de 2 minutes de bout en bout).

c. Infrastructure as Code — Terraform et Ansible

L'infrastructure et la configuration des serveurs sont entièrement automatisées via deux outils complémentaires : Terraform pour le provisionnement des ressources cloud, et Ansible pour la gestion de la configuration logicielle des nœuds.

1. Terraform — Provisionnement AWS

Terraform automatise la création de l'ensemble du réseau et des instances EC2 sur AWS en une seule commande (terraform apply). Le fichier main.tf définit :

- Un VPC (Virtual Private Cloud) pour isoler l'infrastructure.
- Un Security Group avec les règles de pare-feu : ports 22 (SSH), 80 (HTTP), 6443 (API Kubernetes), 30001 (NodePort app).
- Deux instances EC2 Ubuntu 24.04 LTS : Master (t3.small) et Worker (t3.micro).
- Des outputs affichant automatiquement les adresses IP publiques des instances.

```
# Configuration du fournisseur AWS
provider "aws" { region = "us-east-1" }

# VPC et réseau
resource "aws_vpc" "main" {
  cidr_block = "10.0.0.0/16"
  tags = { Name = "DevOps-VPC" }
}

# Security Group — ports K8s, SSH et app
resource "aws_security_group" "k8s_sg" {
  ingress { from_port = 22    to_port = 22    protocol = "tcp" cidr_blocks =
["0.0.0.0/0"] }
  ingress { from_port = 6443  to_port = 6443  protocol = "tcp" cidr_blocks =
["0.0.0.0/0"] }
  ingress { from_port = 30001 to_port = 30001 protocol = "tcp" cidr_blocks =
["0.0.0.0/0"] }
}

# Instance EC2 Master Node
resource "aws_instance" "master" {
  ami           = "ami-04a81a99f5ec58529"  # Ubuntu 24.04 LTS
  instance_type = "t3.small"
  tags          = { Name = "K8s-Master" }
}

# Instance EC2 Worker Node
resource "aws_instance" "worker" {
  ami           = "ami-04a81a99f5ec58529"
  instance_type = "t3.micro"
  tags          = { Name = "K8s-Worker" }
}

# Outputs : adresses IP publiques
```

```
output "master_ip" { value = aws_instance.master.public_ip }
output "worker_ip" { value = aws_instance.worker.public_ip }
```

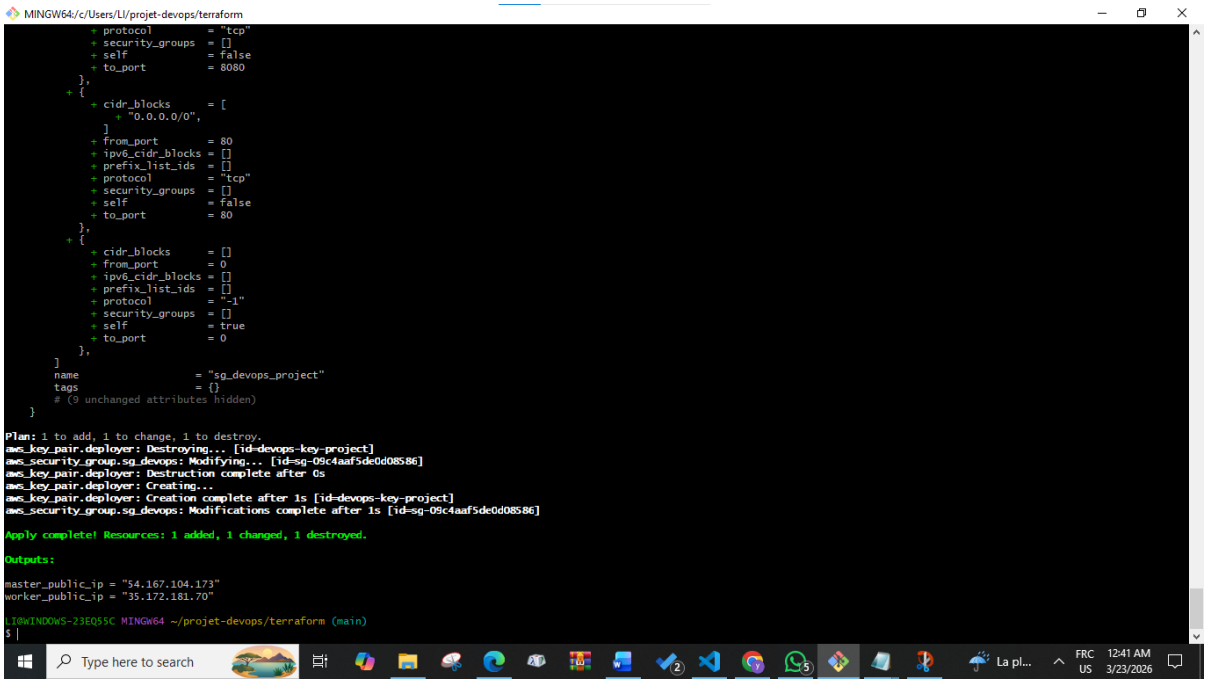


Figure 3 — Résultat de terraform apply : création des instances EC2 et IP publiques

Cette capture d'écran illustre le résultat de l'exécution de terraform apply. Le terminal affiche le déroulement des opérations en temps réel : création du VPC, des Security Groups, des sous-réseaux et des deux instances EC2 Ubuntu 24.04 LTS. Le message "Apply complete! Resources: X added" en bas confirme que toutes les ressources ont été créées sans erreur. Les outputs affichent automatiquement les adresses IP publiques du Master (t3.small) et du Worker (t3.micro), prêtes à être utilisées dans le fichier d'inventaire Ansible.

2. Ansible — Configuration des nœuds

Ansible intervient après Terraform pour configurer logiciellement les instances EC2. Le playbook setup.yml automatise trois étapes critiques en séquence : installation de Docker sur tous les nœuds, déploiement de K3s sur le Master, et jonction sécurisée du Worker au cluster.

```
# ansible/hosts.ini — Inventaire des machines
[masters]
master ansible_host=<IP_MASTER> ansible_user=ubuntu
ansible_ssh_private_key_file=./ma-cle-devops

[workers]
worker ansible_host=<IP_WORKER> ansible_user=ubuntu
ansible_ssh_private_key_file=./ma-cle-devops
```

```
# ansible/setup.yml — Playbook de configuration
```

```yaml
# Étape 1 : Docker sur tous les nœuds
- name: Installer Docker sur tous les serveurs
  hosts: all
  become: yes
  tasks:
    - name: Installation de Docker
      apt: name=docker.io state=present update_cache=yes

# Étape 2 : K3s Server sur le Master
- name: Installer le Master Kubernetes (K3s)
  hosts: masters
  become: yes
  tasks:
    - name: Installer K3s Server
      shell: curl -sfL https://get.k3s.io | sh -
    - name: Récupérer le token K3s
      shell: cat /var/lib/rancher/k3s/server/node-token
      register: k3s_token

# Étape 3 : K3s Agent sur le Worker
- name: Joindre le Worker au cluster
  hosts: workers
  become: yes
  tasks:
    - name: Installer K3s Agent
      shell: |
        curl -sfL https://get.k3s.io | \
        K3S_URL=https://{{ master_ip }}:6443 \
        K3S_TOKEN={{ k3s_token.stdout }} sh -
```

Comparaison Terraform vs Ansible :

Critère	Terraform	Ansible
Rôle principal	Créer/Détruire des ressources cloud	Configurer des machines existantes
Langage	HCL (HashiCorp Config. Language)	YAML (Playbooks)
État	Stateful (terraform.tfstate)	Stateless (idempotent)
Portée	Infrastructure (VMs, réseau, DNS)	Logiciel (Docker, K3s, packages)
Dans ce projet	Création VPC + 2 EC2 sur AWS	Install Docker + K3s + jonction Worker

d. Orchestration Kubernetes — Manifests YAML

L'orchestration de l'application est gérée de manière déclarative via un fichier YAML unique (k8s/deployment.yaml). Ce fichier définit deux ressources Kubernetes essentielles : le Deployment (gestion du cycle de vie des conteneurs) et le Service (exposition réseau de l'application).

Manifest Kubernetes complet

```yaml
# k8s/deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: quiz-deployment
  labels:
    app: quiz-app
spec:
  replicas: 2
  # 2 réplicas = haute disponibilité
  selector:
    matchLabels:
      app: quiz-app
  template:
    metadata:
      labels:
        app: quiz-app
    spec:
      containers:
      - name: quiz-container
        image: chaimaa00dsbd/quiz-app:latest
        imagePullPolicy: Always
        # Always = force pull à chaque redémarrage
        ports:
        - containerPort: 5000
---
apiVersion: v1
kind: Service
metadata:
  name: quiz-service
spec:
  type: NodePort
  selector:
    app: quiz-app
  ports:
  - port: 80
    targetPort: 5000
    nodePort: 30001
    # Accessible via http://IP_AWS:30001
```

Analyse des choix techniques

Paramètre	Valeur choisie	Justification
replicas	2	Haute disponibilité : si un Pod tombe, l'autre reste actif
imagePullPolicy	Always	Kubernetes re-télécharge l'image à chaque redémarrage → CI/CD garanti
Service type	NodePort	Exposition directe sur IP publique AWS sans Load Balancer payant
nodePort	30001	Port personnalisé > 30000 (plage autorisée Kubernetes)
Selector	app: quiz-app	Couplage dynamique Pod/Service sans changer l'adresse d'accès

Vérification du cluster

```
# Vérifier les nœuds du cluster
sudo kubectl get nodes

# Lister les Pods en cours d'exécution
kubectl get pods -o wide

# Détails du service et port exposé
kubectl describe service quiz-service

# Vue globale de toutes les ressources
sudo kubectl get all
```

```
ubuntu@ip-172-31-25-248:~$ sudo kubectl get nodes
NAME                STATUS     ROLES           AGE     VERSION
ip-172-31-18-255    NotReady   <none>          2d20h   v1.34.5+k3s1
ip-172-31-25-248    Ready      control-plane   2d20h   v1.34.5+k3s1
ubuntu@ip-172-31-25-248:~$
```

Figure 4 — Cluster K3s : résultat de kubectl get nodes

Cette capture montre la sortie de la commande kubectl get nodes exécutée sur le Master. Les deux nœuds apparaissent avec le statut "Ready", confirmant que le cluster K3s est pleinement opérationnel. Le Master affiche le rôle "control-plane,master" et le Worker affiche "worker". La colonne VERSION indique la version K3s installée par Ansible, et AGE confirme la stabilité du cluster depuis son déploiement.

e. Monitoring et Observabilité

1. Contraintes techniques et justification

Dans le cadre de l'utilisation du Free Tier AWS (instances t3.micro disposant de 1 Go de RAM), l'installation d'une stack complète Prometheus / Grafana n'a pas été retenue. Ces outils consomment une part importante de la mémoire disponible, ce qui aurait causé une instabilité du cluster Kubernetes K3s.

> **Décision technique**
> *En DevOps, la gestion des ressources est primordiale. La stabilité du service a été priorisée sur la richesse des outils de monitoring. Cette contrainte est documentée et une roadmap est proposée pour une infrastructure de production.*

2. Monitoring natif déployé

Outil	Commande / Méthode	Usage
kubectl logs	kubectl logs -f [POD_NAME]	Suivi des requêtes HTTP Flask en temps réel
kubectl get nodes	kubectl get nodes	État des nœuds Master et Worker
kubectl get pods	kubectl get pods -o wide	Répartition et état des Pods
kubectl describe	kubectl describe pod [NAME]	Diagnostic des erreurs de démarrage
GitHub Actions	Tableau de bord GitHub	Temps de build, taux de succès CI/CD

3. Architecture cible (Roadmap Production)

Pour une infrastructure de production disposant de ressources adaptées (minimum 4 Go de RAM par nœud), la solution de monitoring envisagée serait :

- Prometheus — collecte des métriques système (CPU, RAM, requêtes/s, latence).
- Grafana — dashboards interactifs pour la visualisation des métriques.
- Loki — agrégation centralisée des logs applicatifs.
- Promtail — agent de collecte et de forwarding des logs vers Loki.

IV. Outils et Justification des Choix Technologiques

Le choix de chaque outil a été motivé par des critères techniques précis, adaptés aux contraintes du Free Tier AWS et aux objectifs pédagogiques du projet.

Catégorie	Outil	Version	Justification du choix
CI/CD	GitHub Actions	v3	Légèreté : contrairement à Jenkins auto-hébergé, le build s'exécute sur les serveurs GitHub et ne consomme pas la RAM des instances AWS.
Conteneurisation	Docker	24.x	Standard industriel. Garantit que l'app tourne identiquement en dev et en prod. Base de l'image : python:3.9-slim.
Orchestration	K3s (Kubernetes)	v1.28	Version certifiée CNCF, légère (512 MB RAM vs 2 GB pour kubeadm), idéale pour les instances t3.micro. Binaire unique.
IaC Cloud	Terraform	v1.5+	Gestion du cycle de vie des ressources AWS (créer, modifier, détruire) via code HCL versionnable et reproductible.
Config Mgmt	Ansible	v2.12+	Agentless (pas d'agent à installer), idempotent, simple YAML. Configuration automatique de Docker et K3s sur les nœuds EC2.
Cloud Provider	AWS EC2	us-east-1	Free Tier disponible : 750h/mois de t3.micro. Région us-east-1 : latence faible, richesse des AMIs disponibles.
Registry	Docker Hub	Free	Registry public gratuit. Intégration native GitHub Actions. Image accessible depuis les nœuds EC2 sans configuration réseau spéciale.

V. Preuves de Fonctionnement

Cette section présente les captures d'écran prouvant le bon fonctionnement de chaque composant de l'infrastructure déployée.

1. Infrastructure AWS — Instances EC2

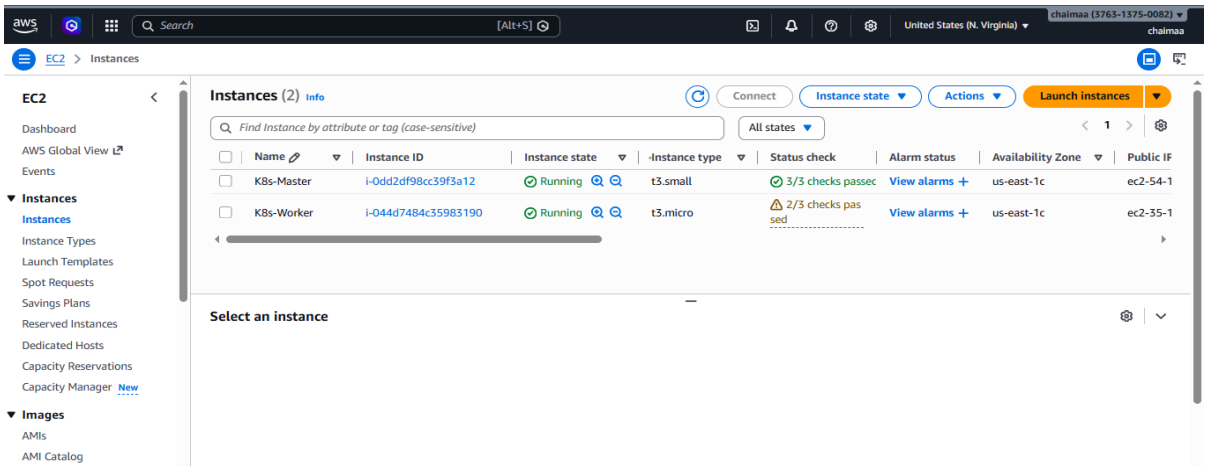
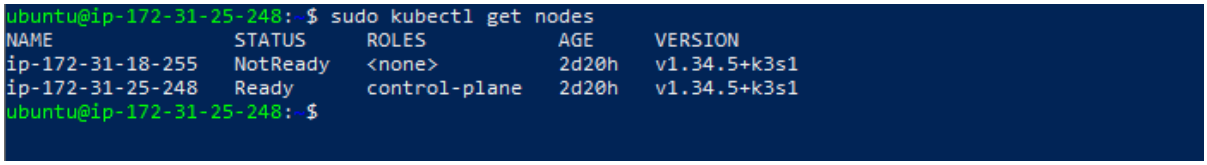


Figure 5 — Console AWS EC2 : instances K8s-Master et K8s-Worker en état "Running"

Cette capture de la console AWS EC2 confirme que les deux instances créées par Terraform sont actives dans la région us-east-1. Le statut "Running" indique que les machines virtuelles sont en cours d'exécution. Les colonnes "Instance state" et "Status check" affichent respectivement "Running" et "2/2 checks passed", certifiant que les instances sont accessibles depuis le réseau et que les diagnostics système sont satisfaisants. Les tags Terraform (K8s-Master et K8s-Worker) permettent de les identifier clairement.

2. Cluster Kubernetes (K3s)



```
ubuntu@ip-172-31-25-248:~$ sudo kubectl get nodes
NAME                STATUS     ROLES           AGE     VERSION
ip-172-31-18-255    NotReady   <none>          2d20h   v1.34.5+k3s1
ip-172-31-25-248    Ready      control-plane   2d20h   v1.34.5+k3s1
ubuntu@ip-172-31-25-248:~$
```

Figure 6 — Cluster K3s : les deux nœuds enregistrés avec statut "Ready"

Cette capture montre la sortie de kubectl get nodes après l'exécution du playbook Ansible. Les deux nœuds — K8s-Master (Control Plane) et K8s-Worker (Data Plane) — apparaissent dans le même cluster avec le statut "Ready". Ce résultat prouve que l'automatisation Ansible a correctement installé K3s sur le Master, récupéré le token de jonction, et configuré le Worker pour rejoindre le cluster. La communication inter-nœuds via le réseau VPC AWS est pleinement fonctionnelle.

3. Pipeline CI/CD — GitHub Actions

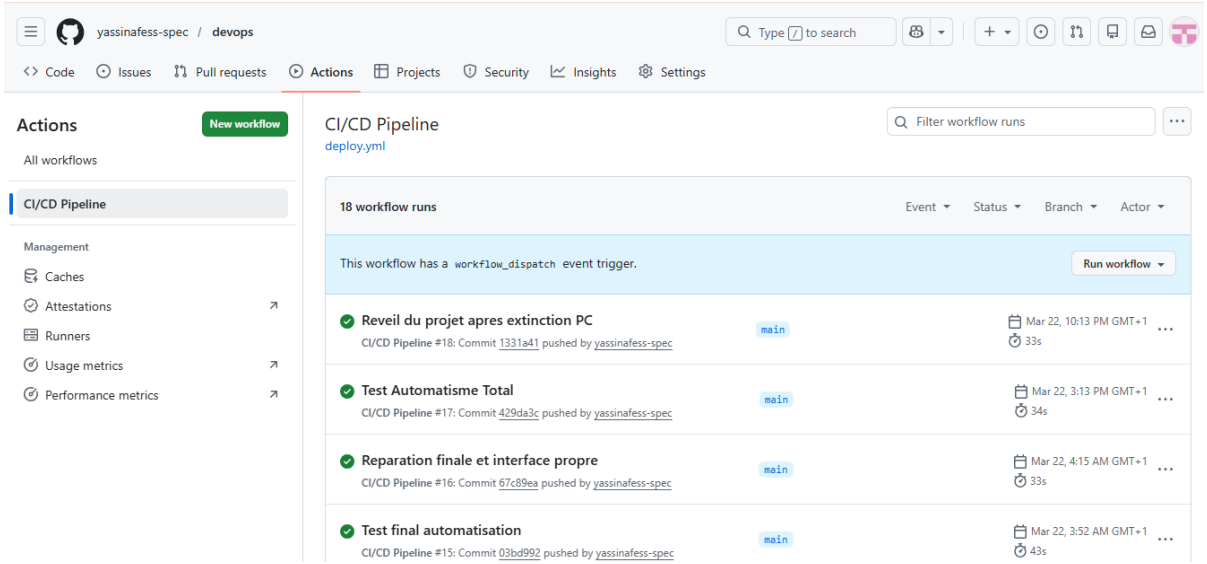


Figure 7 — Pipeline GitHub Actions : toutes les étapes en succès (statut vert)

Cette capture confirme le fonctionnement complet du pipeline CI/CD. Chaque run GitHub Actions déclenché par un git push sur la branche main a exécuté avec succès l'intégralité des étapes : construction de l'image Docker avec --no-cache pour garantir la fraîcheur du code, push de l'image vers le registry Docker Hub avec un tag unique, connexion SSH sécurisée au Master EC2 via la clé privée stockée dans les GitHub Secrets, et mise à jour des Pods Kubernetes avec kubectl rollout restart assurant un déploiement sans interruption de service.

4. Image Docker — Docker Hub

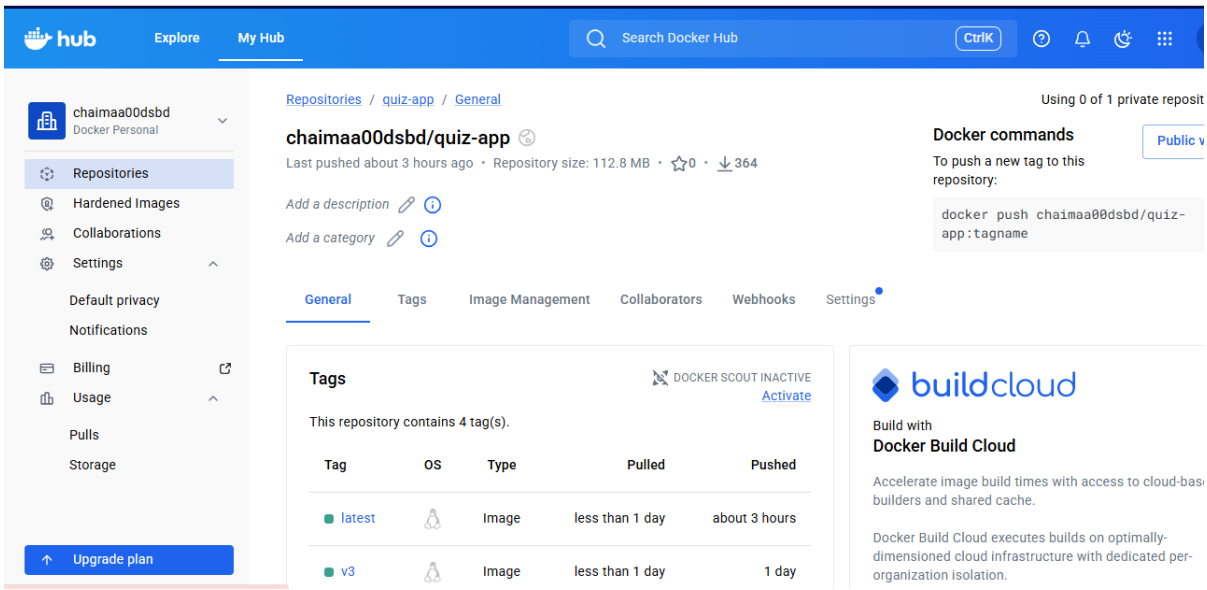


Figure 8 — Docker Hub : image quiz-app:latest publiée automatiquement par le pipeline

Cette capture de l'interface Docker Hub montre le repository de l'image de l'application Quiz. Le tag "latest" a été mis à jour automatiquement par GitHub Actions lors du dernier push. Les métadonnées affichent la date de dernière mise à jour, la taille de l'image compressée (reflétant l'efficacité de

l'image de base python:3.9-slim), et le nombre de pulls. Cette image est ensuite téléchargée par les nœuds Worker du cluster Kubernetes grâce à la directive imagePullPolicy: Always définie dans le manifest Kubernetes.

5. État du cluster (kubectl)

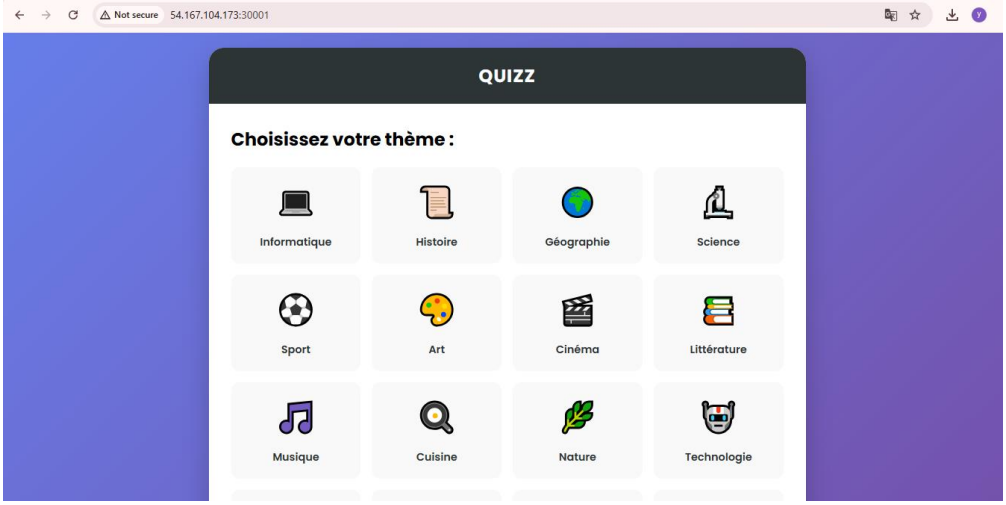


Figure 9 — kubectl get all : vue complète des ressources Kubernetes actives

Cette capture affiche la sortie de la commande kubectl get all, donnant une vue synthétique de l'état complet du cluster Kubernetes. On y distingue le Deployment quiz-deployment avec ses 2 réplicas désirés et disponibles (READY: 2/2), les deux Pods en état "Running" sur le nœud Worker, le Service quiz-service de type NodePort mappant le port 30001 vers le port 5000 des conteneurs Flask, et le ReplicaSet gérant automatiquement la haute disponibilité. L'ensemble de ces ressources a été créé à partir du seul fichier k8s/deployment.yaml.

Figure 10 — kubectl get pods -o wide : répartition des Pods sur le cluster

Cette capture montre la sortie de kubectl get pods -o wide, affichant les informations détaillées de chaque Pod en cours d'exécution. La colonne NODE confirme que les deux Pods (quiz-app) sont schedulés sur le nœud Worker, conformément à la stratégie de déploiement K3s. La colonne IP

affiche l'adresse IP interne attribuée par le réseau overlay K3s (Flannel). La colonne STATUS indique "Running" pour les deux Pods, et RESTARTS affiche 0, confirmant une exécution stable sans redémarrage non planifié.

VI. Guide de Reproduction (Runbook)

Ce guide décrit les étapes séquentielles pour reproduire intégralement l'infrastructure depuis zéro sur un nouveau compte AWS.

Prérequis

Outil	Version minimale	Installation
Terraform	v1.5+	https://developer.hashicorp.com/terraform/install
Ansible	v2.12+	pip install ansible
AWS CLI	v2.x	https://aws.amazon.com/cli/ + aws configure
Git	v2.x	Pré-installé sur Linux/Mac
Docker Hub	Compte gratuit	https://hub.docker.com/

Étape 1 — Provisionnement de l'Infrastructure (Terraform)

Générer une paire de clés SSH pour accéder aux instances EC2 :

```
ssh-keygen -t rsa -b 4096 -f ./ma-cle-devops
```

Initialiser Terraform et déployer les ressources AWS :

```
cd terraform
terraform init          # Téléchargement du provider AWS
terraform plan          # Simulation — valider avant exécution
terraform apply -auto-approve  # Déploiement réel
```

Récupérer les adresses IP publiques affichées dans la console (outputs Terraform).

> **Gestion des coûts**
> *AWS Free Tier offre 750h/mois de t3.micro. Pensez à exécuter "terraform destroy -auto-approve" lorsque vous ne travaillez pas sur le projet pour éviter toute facturation.*

Étape 2 — Configuration du Cluster (Ansible)

Mettre à jour ansible/hosts.ini avec les adresses IP obtenues à l'étape précédente, puis lancer :

```
cd ../ansible
ansible-playbook -i hosts.ini setup.yml
```

Vérifier la santé du cluster depuis le Master :

```
ssh -i ../ma-cle-devops ubuntu@<IP_MASTER> "sudo kubectl get nodes"
```

Étape 3 — Configuration CI/CD (GitHub Actions)

Ajouter les secrets dans Settings → Secrets and variables → Actions :

Secret	Valeur
DOCKER_USERNAME	Pseudo Docker Hub
DOCKER_PASSWORD	Mot de passe ou Access Token Docker Hub
IP_MASTER	Adresse IP publique du Master Node (output Terraform)
SSH_PRIVATE_KEY	Contenu complet du fichier ./ma-cle-devops

Déclencher le premier déploiement :

```
git add .
git commit -m "feat: initial deployment"
git push origin main
```

Étape 4 — Accès à l'Application

L'application est accessible publiquement via :

```
http://<IP_MASTER>:30001/
```

Exemple avec l'IP de ce projet : http://54.167.104.173:30001/

Étape 5 — Nettoyage des Ressources

```
cd terraform
terraform destroy -auto-approve
```

Cette commande supprime l'intégralité des ressources AWS créées (EC2, VPC, Security Group) en évitant toute facturation inattendue.

VII. Documentation et Debugging

Validation avec terraform plan

En DevOps, on ne crée jamais de serveurs "à l'aveugle". La commande terraform plan est une simulation sèche qui permet de visualiser exactement ce que Terraform va créer, modifier ou supprimer, sans toucher à l'infrastructure réelle.

- Ressources vérifiées : nombre d'éléments à ajouter (ex : 4 à ajouter).
- Syntaxe validée : détection des erreurs dans les fichiers .tf avant exécution.
- Sécurité : vérification des ports ouverts, des politiques IAM et des CIDR.
- Drift detection : comparaison entre l'état souhaité et l'état réel (terraform.tfstate).

Commandes de diagnostic Kubernetes

Commande	Usage
kubectl get nodes	Vérifier que Master et Worker sont en statut Ready
kubectl get pods -o wide	Répartition des conteneurs et nœud d'exécution
kubectl logs [POD]	Lire les logs Flask en cas de plantage applicatif
kubectl describe pod [POD]	Diagnostiquer pourquoi une image ne se télécharge pas
kubectl rollout status	Suivre l'avancement d'un déploiement en cours
kubectl rollout restart	Forcer la mise à jour des Pods (utilisé dans CI/CD)
sudo kubectl get all	Vue globale : Pods, Services, Deployments, ReplicaSets

Problèmes rencontrés et solutions apportées

Problème	Cause	Solution
Image non mise à jour	Kubernetes gardait l'ancienne image en cache	Ajout de imagePullPolicy: Always dans le manifest
Worker en NotReady	Service k3s-agent arrêté après redémarrage	sudo systemctl restart k3s-agent sur le Worker
Port 30001 inaccessible	Security Group AWS manquant la règle	Ajout de l'ingress port 30001 dans main.tf Terraform
RAM insuffisante	Prometheus/Grafana trop lourds pour t3.micro	Monitoring natif kubectl + roadmap production

VIII. Conclusion

Ce projet a permis de mettre en œuvre une infrastructure DevOps complète et fonctionnelle, couvrant l'ensemble du cycle de vie d'une application : de l'écriture du code jusqu'à son exposition aux utilisateurs finaux sur une infrastructure cloud AWS, entièrement automatisée.

L'objectif principal a été pleinement atteint : chaque commit déclenche automatiquement un pipeline CI/CD qui construit l'image Docker, la publie sur Docker Hub et met à jour les Pods Kubernetes sur AWS, sans aucune intervention manuelle et sans interruption de service (zéro downtime).

Compétences acquises

Domaine	Compétence maîtrisée
Infrastructure as Code	Provisionnement AWS avec Terraform (VPC, EC2, Security Groups)
Gestion de configuration	Automatisation avec Ansible (Docker, K3s, inventaire)
Conteneurisation	Création d'images Docker optimisées (python:3.9-slim)
CI/CD	Pipeline GitHub Actions complet avec déploiement Kubernetes
Orchestration	Cluster K3s avec haute disponibilité (2 réplicas, rollout)
Sécurité	Gestion des secrets GitHub, clés SSH, Security Groups AWS
Debugging	Utilisation de kubectl, terraform plan, logs applicatifs

Perspectives d'amélioration

- Monitoring avancé : déploiement de Prometheus + Grafana + Loki sur instances plus puissantes.
- Sécurité renforcée : RBAC Kubernetes, Network Policies, scan de vulnérabilités Docker (Trivy).
- Haute disponibilité avancée : ajout d'un Ingress Controller et d'un Load Balancer AWS (ALB).
- GitOps : adoption d'ArgoCD ou Flux pour un déploiement déclaratif synchronisé avec Git.
- Tests automatisés : intégration de tests unitaires et d'intégration dans le pipeline CI.

Bilan global

Ce projet démontre la maîtrise des outils DevOps modernes dans un contexte cloud réel. L'infrastructure est entièrement reproductible depuis zéro via le Runbook documenté, répondant pleinement aux exigences du projet de Master DSBD & IA.

Annexes

Annexe A — Liens et Ressources

Ressource	Lien
Repository GitHub	https://github.com/yassinafess-spec/devops
Image Docker Hub	https://hub.docker.com/r/chaimaa00dsbd/quiz-app
Application déployée	http://54.167.104.173:30001/
Documentation K3s	https://docs.k3s.io/
Documentation Terraform AWS	https://registry.terraform.io/providers/hashicorp/aws/
Documentation Ansible	https://docs.ansible.com/
GitHub Actions Docs	https://docs.github.com/en/actions

Annexe B — Récapitulatif des Fichiers Livrés

Fichier	Emplacement	Description
app.py	app/	Code source Flask — API Quiz
requirements.txt	app/	Dépendances Python
Dockerfile	app/	Image conteneur Python:3.9-slim
main.tf	terraform/	Provisionnement AWS (VPC, EC2, SG)
hosts.ini	ansible/	Inventaire des nœuds EC2
setup.yml	ansible/	Playbook Docker + K3s
deployment.yaml	k8s/	Manifest Kubernetes (Deployment + Service)
deploy.yml	.github/workflows/	Pipeline CI/CD GitHub Actions